Agentic System Design Process

Nine phases from problem to production, assembled from the course materials and practitioner guides

J. A. Arroyo

Version 1.12 — 7 November 2026 (appendix diagrams: one gateway per cap, the test for verified)

Abstract

I did not find a complete lifecycle process in the public materials reviewed from the fifteen courses indexed in the guide [26]. Individual courses cover some phases explicitly, through project templates, rubrics and slides, and the practitioner guides from Anthropic, OpenAI and Google cover most of the rest. Version 1.10 adds practices from official provider documentation and recent papers, read in full on 2 November 2026. This document combines those sourced practices with my synthesis. **The phase order, the gates and the resolutions between sources are inferred recommendations; the citations identify support for individual practices.** Places where the sources differ are stated, not hidden. [inferred] marks a statement that the cited sources do not explicitly prescribe.

Contents

How to use this document 2

Phase 1: Frame the problem 2

Phase 2: Decide whether an agent is warranted 3

Phase 3: Decompose the task and assign ownership 5

Phase 4: Define the evaluation 6

Phase 5: Design tools, state and context 7

Phase 6: Build a minimal prototype and select models 8

Phase 7: Evaluate, diagnose and iterate 9

Phase 8: Harden safety and approve release 11

Phase 9: Deploy gradually and maintain 11

Different emphases and my design choices 13

Limits of this document 13

Authorship and use of AI tools 13

Appendix: Worked examples 14

Sources 18

How to use this document

One set of three documents

1. **Learning to Build Agentic Systems** (the guide): the courses, lectures and assignments behind the practices. *Use it to learn a topic.*
2. **Agentic System Design Process**: nine phases from framing a problem to running it in production. *Use it to design a system.*
3. **Screening Tasks for Agentic Automation**: a one-page worksheet, filled in after Phase 1 of the design process, that feeds Phase 2 and sets up the comparison run in Phase 7. *Use it to decide whether a task is worth it.*

This is document 2. The courses it cites are indexed in the guide; the worksheet it uses in Phase 2 is in the screening companion [27].

Work the phases in order, and write down the output of each phase before moving on. The process is not linear in practice:

- Between Phase 1 and Phase 2, fill in the screening worksheet [27, Section 6]. Its answers are the input to Phase 2, and its last row is the comparison that Phase 7 runs.
- Phase 7 can send you back to Phase 2 (simplify) or Phase 3 (re-split the work).
- Phase 9 triages production failures and adds representative regression cases to the Phase 4 set.
- Safety is not a late add-on. It is designed in Phases 1, 3 and 5 and tested in Phase 8.

Scale each phase to the stakes. For a small internal tool, a phase can be a paragraph; for a customer-facing system, it is a reviewed document. The appendix works three examples through the worksheet and the phases.

#	Phase	Output	Support
1	Frame the problem	Problem brief with success and unacceptable failures	Strong
2	Decide whether an agent is warranted	Option comparison; written justification	Strong (guides)
3	Decompose and assign ownership	Task map: code / model / human; stop and approval points	Strong
4	Define the evaluation	Test set, metrics, baseline, release bar	Very strong
5	Design tools, state and context	Tool contracts, state model, permission boundaries	Strong
6	Prototype and select models	One traced working path; model comparison	Practices strong; order disputed
7	Evaluate, diagnose, iterate	Error analysis; improvements vs. baseline	Very strong
8	Harden safety and approve release	Adversarial results; release sign-off	Strong
9	Deploy gradually and maintain	Versioned rollout, monitoring, regression growth	Strong (guides); weak (courses)

Figure 1 shows the phases, the return paths and where safety work happens.

Phase 1: Frame the problem

State what problem is being solved, for whom, and how you will know it is solved, before any design choice is made.

Questions to answer

- Who is the user, and what task are they trying to complete?
- How is it done today, and what is wrong with that? Why does it matter?
- What data or inputs are available, and what constraints apply (cost, latency, privacy, regulation, deployment environment)?
- What observable result counts as success?

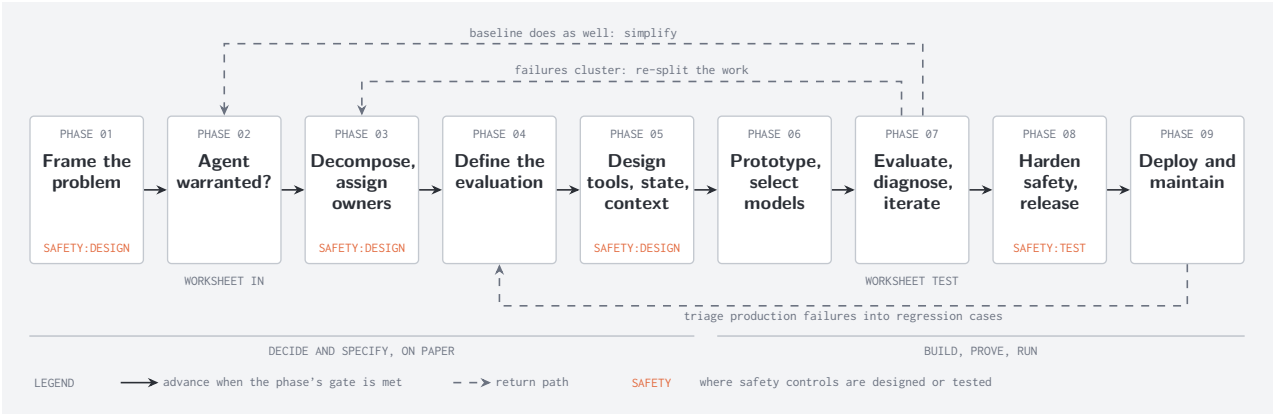


Figure 1: The nine phases. Return paths and safety touchpoints follow the text; the order and the gates are my synthesis [inferred]. The screening worksheet [27] is filled in before Phase 2, and its comparison is run in Phase 7.

- Which failures are unacceptable (wrong irreversible action, data leak, confident wrong answer)? These seed the safety work in Phases 3, 5 and 8.
- For each class of unacceptable failure, what does it trigger: a block on release, a referral to a person, or a rollback? Who may pause or switch the system off? NIST's AI RMF playbook asks that responsibility to supersede, disengage or deactivate a system be assigned, with thresholds for doing so [50, MANAGE 2.3–2.4]. OpenAI rates each tool low, medium or high risk and uses the rating to pause for checks or escalate to a person [5, p. 26]. Phases 8 and 9 apply this map [inferred].

Evidence. Colorado's project proposal asks for the problem, why it matters, the state of the art and its limits, the proposed solution, and how you would convince a reader it works [29]. Mannheim's project outline requires four answers: the problem, the data, the method including which LLMs, and how success is measured [28, slide 18]. Buffalo's final project begins with a problem statement and an environment and tool inventory [23]. OpenAI and Google both start from the use case and its business problem [5, pp. 5–6][6, pp. 19–20].

Done when

A one-page brief exists that names the user, the task, the constraints, the success measure, and the unacceptable failures with what each one triggers.

Phase 2: Decide whether an agent is warranted

Choose the simplest option that can meet the Phase 1 success measure, and justify in writing every step up in complexity.

Input. The Phase 1 brief and the completed screening worksheet [27, Section 6]. Row S records how work starts. Rows 0a and 0b test the simpler alternatives; rows A1 to A3 assess further model-assisted automation when those alternatives are not enough; rows B1 to B4 identify the control decisions left unresolved and plan the comparison.

Before the ladder: alternatives without a language model. First ask whether a conventional statistical or machine-learning model, or a numerical optimizer, would meet the success measure, alone or alongside a script. Record the strongest such option and the evidence that it meets or misses the target; if that is unknown, run the smallest test that would tell you. These are not a rung of the ladder, because they can also serve as tools at any rung [inferred]. Mannheim's non-LLM baseline is the closest course precedent [28, slide 26]; the screening companion gives the cases behind this check [27, Section 3].

The ladder (Figure 2). Consider the options in this order and stop at the first one that is adequate [inferred]:

1. **Deterministic code** (rules, scripts, a form). Predictable, cheap, testable.
2. **A single LLM call**, optionally with retrieval and examples.

3. **A fixed workflow**: several LLM calls and code steps in a path set in advance (chaining, routing, parallelisation).
4. **A bounded agent**: the model chooses the next step in a loop, with tools, limits and stopping rules.

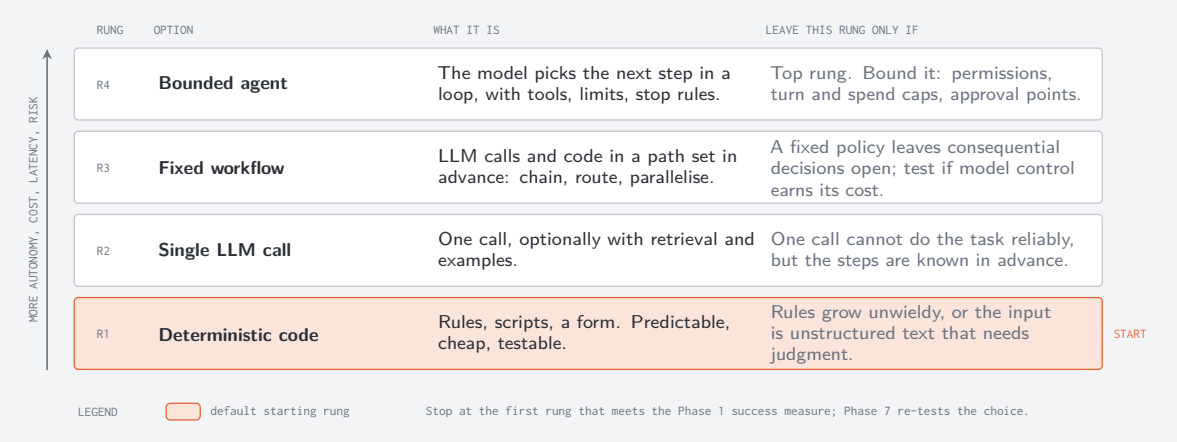


Figure 2: The Phase 2 ladder. The exit conditions paraphrase Anthropic [1, "When (and when not) to use agents"], OpenAI [5, pp. 5–6] and Northeastern [20, slides 9, 10, 14]; arranging them as a ladder, and the tighter wording of the exit from the fixed workflow, are my synthesis [inferred].

Criteria for moving up.

- Anthropic: find the simplest solution possible and add complexity only when it demonstrably improves results; workflows suit well-defined tasks, agents suit open-ended problems where the steps cannot be predicted, at the price of cost, latency and compounding errors [1, "When (and when not) to use agents"].
- OpenAI: consider an agent when the task needs complex judgment, the rule set has become unwieldy, or it relies heavily on unstructured data; otherwise a deterministic solution may suffice [5, pp. 5–6].
- Northeastern distinguishes fixed workflows and one-time routing from a loop in which the model selects actions using their results. Cycles alone are insufficient: the model must direct the next action [20, slides 9, 10, 14].
- CMU places developer-defined workflows, plan-then-execute and adaptive agents on one spectrum: a fixed plan is faster, cheaper and more predictable, and replanning should be bought only when its feedback is worth the cost [15, p. 16].
- Stanford CS146S, for skills: a single fixed command is a shell alias, not a skill; a skill needs repetition and a real decision point [18, Part I].

Leaving the fixed workflow. "The steps cannot be predicted" is not enough to leave the fixed-workflow rung, because a fixed workflow can branch, retry and escalate. Write the strongest fixed policy you can maintain, including those branches, and list the decisions it leaves unresolved. No branch may hide an agent: "diagnose and repair" is not a step unless the policy says how. Move up only if those unresolved decisions are worth more than the added cost and risk [inferred].

Before authorizing a bounded agent, answer three questions about feedback [inferred]:

- What observation will guide the model's next decision during a run?
- What checks success independently of the model, in a way the agent cannot alter?
- How has that check been tested, for example on cases already known to be good and bad?

Weak or misleading feedback can misdirect the agent's adaptation. Record its limitations and test whether it is informative enough for the decisions the agent will make [inferred]. The screening companion gives the cases behind these feedback questions and the exit from the fixed workflow [27, Sections 3–4].

Gap in the courses. In the public material checked, no course asks students to write down why a script or a single call would not do. Only the Anthropic and OpenAI guides state the test. This phase makes it an explicit gate. The decision is provisional: Phase 7 confirms it empirically by comparing against the simpler option, as CS329A HW3 does when it builds a single-call baseline before adding routing, decomposition and refinement [32, Parts 0–3], and as Mannheim does with a non-LLM baseline [28, slide 26].

Done when

The chosen option is recorded, with the evidence for rejecting each relevant simpler alternative and any remaining unknowns. If the provisional choice is a bounded agent, the record also holds the strongest maintainable fixed policy, the decisions it appears to leave unresolved, their expected value, the answers to the three feedback questions and the planned comparison.

Phase 3: Decompose the task and assign ownership

Split the work into tasks and subtasks, and decide for each whether code, the model or a human owns it.

Allocation rules (Northeastern, week 3b, rules 1–6 [19]; Figure 3):

- Policy, calculations, permissions and budgets live in **code**; contextual judgment and tool selection live in the **model**. "The model proposes; the runtime disposes."
- Prompt rules that read like code belong in code.
- Tools validate their own inputs, obtain their own prerequisites, and return both facts and verdicts, so they cannot be talked out of a rule.

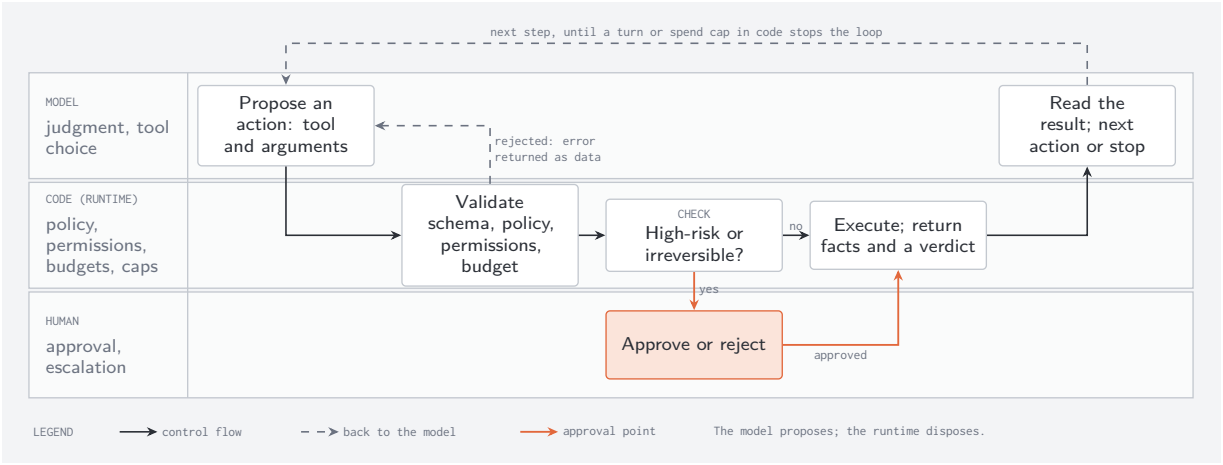


Figure 3: Ownership for one step of an agent loop: the model proposes, code validates and executes, and a human approves high-risk actions. Allocation follows Northeastern [19, rules 1–6]; human intervention on high-risk actions follows OpenAI [5, pp. 24–31]. The sequence is an illustration [inferred].

Design checklist (CMU lecture 5, p. 46 [15]):

- Are the subtasks really separable, and would different parts benefit from different models?
- Which observations should change the plan?
- How will errors and context be kept from piling up?
- What information or authority must stay separate, and who approves irreversible actions?

Also record: stopping conditions (turn and spend caps), the points where a human must approve, and the escalation path when the agent is unsure. LangGraph's method maps the workflow into discrete steps and labels

each as an LLM step, a data step, an action or a human-input step [9, steps 1–2]. OpenAI requires human intervention on failure thresholds and high-risk actions [5, pp. 24–31].

Single or multiple agents. OpenAI recommends maximising a single agent first and considering a split when complex instructions or overlapping tools impair performance [5, p. 16]. Anthropic describes additional patterns for parallel work and dynamic delegation [1, "Parallelization", "Orchestrator-workers"]. Berkeley's Lab 2 recommends planning, generation and verification agents for its Lean task [37]; treat that as a design for that task, not a default.

Delegating to another agent. Write a brief for each delegated task: objective, inputs, output format, tools and sources, bounds, how the result will be checked, and whom it reports to. Anthropic found that vague briefs led subagents to duplicate work and leave gaps [54, "Prompt engineering and evaluations for research agents"]. Grant only the permissions the task needs, within the delegator's own. DeepMind proposes splitting work whose result cannot be checked until it can, choosing the response to a failure by how reversible and how consequential the task is, and attenuating privileges at each hand-off; this is a proposed framework, not a tested method [56, §4.1, p. 9; §4.4, p. 13; §4.7, p. 18]. A verification agent needs explicit completion criteria or it takes shortcuts [55, "The verification subagent pattern"], and MAST found that many verifiers in the systems it studied made only superficial checks [57, §4, pp. 7–8]; check both that the result is correct and that it meets the objective [inferred]. Treat a subagent's report as data: instructions or approval claims inside it carry no authority from the user [58, "Subagent output scanning"].

Human control at each step. Record for each step how much the system does alone and what the person sees and can change. PAIR advises automating tasks that are tedious, repetitive or dangerous, and augmenting where the stakes are high or personal responsibility matters [59, "When to automate", "When to augment"]. Escalate when retry or action limits are exceeded and on high-risk actions [5, p. 31]. Show a confidence display only if it changes what the user does, and test the form of display early [60, §4 "Manage influence on user decisions"]. State what each kind of user feedback means and how the system responds to it [61, "Collect explicit feedback", "Review implicit feedback"]. Where the model decides when to ask the user, evaluate that decision too: on HiL-Bench, models that did well with full information recovered only a fraction of it when they had to judge when to ask (best 38% on SQL and 12% on software tasks, against 86–91% and 64–88% with full information) [62, §4.2, Table 1].

Done when

A task map lists every subtask with its owner (code, model, human), the stopping conditions, and the approval points.

Phase 4: Define the evaluation

Decide how success will be measured before building, so the prototype is judged against evidence rather than demos.

Contents

- **A first test set.** Anthropic recommends starting early with 20–50 simple tasks drawn from real failures, because evals get harder to build the longer you wait [3]. Include known-good cases, known failures and edge cases. Berkeley requires test cases with known outcomes, including edge cases, and asks that a human be able to solve each task [35, pp. 82–85, 91]. Those known-outcome cases test the evaluator itself [35, p. 85].
- **Metrics** across quality, cost, latency and safety. Mannheim's method: define success criteria, choose the method (code checks, LLM judge, human review), use tooling, iterate [24, p. 76]. CMU evaluates at four levels: tool selection, arguments, trajectory and final task, plus efficiency, reliability and safety [14, pp. 54–56].
- **Reliability, not only success rate.** Run each task several times to measure consistency; test robustness by injecting tool faults, changing tool-interface formats and paraphrasing prompts; and, if the agent reports confidence, measure its calibration. Rabanser et al. define such metrics in four groups (consistency, robustness, predictability, safety) and used five runs per task, a fault probability of 0.2 and five paraphrases [13, §3, Table 2; §4.1]. Five runs is their experimental setting, not a prescribed minimum; choose the count to balance estimation precision against cost [inferred].

- **The grader.** Prefer code checks, then model graders, with humans to calibrate [3]. An LLM judge is not always needed: verifiable tasks can be checked objectively [35, pp. 17, 71].
- **Development and held-out cases.** Use development cases for iteration and model selection, and keep held-out cases to check generalisation before release. Validate graders against known successes and failures, and reset environment and memory state between independent trials [2, "Collaborating with agents"][3, steps 2, 4, 5].
- **Consistent human labels.** A task is well specified when two domain experts would independently reach the same verdict [3, step 2]. Give labellers examples for each score level and refine the rubric over several review rounds [40, "Human evals"]; include cases that even humans would find hard to agree on [41, "Example edge cases"]. When several people label, measure their agreement and record why each disagreement was settled [inferred]. Agreement shows consistency, not validity: labellers can share the same error [42, p. 27].
- **Tone, where it matters.** Write criteria and examples for it and validate its grader against human judgments [41, "Grade your evaluations"]. Anthropic's example rubric checks whether the agent showed empathy [3, "Evaluating conversational agents"].
- **Simulated users** for multi-turn tasks: give each simulator a persona and a conversation plan suited to the task, and a turn limit [43, "User simulation", "User personas"]. Check how faithful the simulator is and how much results move with the simulator chosen. Agent success varied by nearly 9 points across four simulator models in one study [44, §4.1, Table 1]; in another, simulators were more cooperative and uniform than 451 human participants and inflated success [45, §5]. Confirm claims about real users with human data [inferred].
- **The baseline**: the simpler rung from Phase 2, run on the same set.
- **The release bar**: the threshold, on these metrics, that Phase 8 must confirm. For reliability, consider pass^k (success on all k attempts) rather than pass@k [36, p. 30].

Order dispute. The Anthropic evals post and the Berkeley project put evaluation first (Berkeley's project structure introduces evaluator development as Phase 1 and participant-agent development as Phase 2 [38, Grading, Project Timeline]; both final submissions share a deadline, so this is an emphasis, not a strict gate); the Anthropic tools post prototypes first and then evaluates [2]; OpenAI sets up evals as the baseline for model selection [5, p. 8]. Resolution [inferred]: write a small set before building, then grow it from the failures found in Phase 7.

Done when

A versioned test set, the metrics, the grader for each, the baseline and the release bar are written down.

Phase 5: Design tools, state and context

Specify everything the model can touch and see, so that its actions are constrained by design rather than by prompt wording.

Tools

- Prefer a few workflow-level tools to many thin endpoint wrappers; namespace them; return high-signal, token-efficient results; make errors actionable [2].
- Put constraints in the schema (enums, ranges, required fields), shape the outputs, return errors as data, treat the docstring as the contract, and put a brake on write tools (read-only hints, dry runs) [17, Part II].
- Design the interface to be hard to misuse (poka-yoke), for example requiring absolute paths [1, Appendix 2].
- Call a tool only when its benefit outweighs its latency, cost, failure modes and risk [14, p. 5].

State and context

- Define the state the system carries between steps, how it changes, and what happens on each error path [9, steps 3–4].

- Decide what context each step needs and keep it minimal and high-signal [4]. Persistent memory is a design choice, not a required component.
- For a recurring agent, track progress per source with a stored bookmark rather than a fixed time window. If a source cannot be read, keep its bookmark and say so in the output; never report it as "nothing new" [11, Sources].
- Record a side effect as done only after the destination confirms it, and check for an existing result before retrying to reduce duplicate effects [11, Destination].
- Keep the rules the agent must follow in a store it can read but not edit; have it re-read them every run and stop if it cannot [11, Memory].
- If agents share memory, map each store: owner, scope, who may read and write, provenance, how concurrent writes are handled, and retention and deletion [inferred]. Claude Managed Agents memory stores are read-write by default; Anthropic warns that a successful prompt injection could write malicious content into one and advises read-only access for shared reference material [63, access section]. ADK scopes state by prefix: session, user, app and temporary [64, "Organizing State with Prefixes"]. Margalit et al. name the failure modes to design against: unauthorised leakage, stale propagation, persistent contradictions and lost provenance [65, §4, p. 4].
- Treat curated context as an owned asset. Notion separates data context (metrics, sources, required filters), business context and routing guidance, and assigns owners and review to keep them current [12, "Connecting data and business context", "Keeping data and business context current"].

Permission boundaries. Least privilege for every tool and credential, sandboxed execution, and paths confined to a working directory [30][16]. Minimal OAuth scopes, each with a stated reason [17]. When the agent acts for a user, let it use that user's own credentials so existing access controls apply, and have it report denied access instead of working around it [12, "Governing the agent and its data access", "Clear instructions for repeatable analysis"].

Done when

Every tool has a contract (inputs, outputs, errors, side effects, permission level); the state model and error paths are written; permissions are listed.

Phase 6: Build a minimal prototype and select models

Get one path working end to end with full tracing, then choose models by measuring them on the Phase 4 development set.

Build

- Build the simplest version of the chosen rung. Anthropic suggests starting with direct API calls rather than a framework, so the mechanics stay visible [1]; Google calls a production-grade framework essential [6, p. 23]. Resolution [inferred]: specify the capabilities you need and choose a library only if it supplies them.
- Test each tool on its own before wiring it in, and write the system prompt last [21].
- Trace every run: prompts, tool calls, results, cost and latency. Phase 7 depends on it.
- Before building infrastructure, check the managed options. Microsoft's adoption framework favours the option with the least operational responsibility that meets the requirements [66, §3]. Providers now offer managed agent runtimes alongside SDKs and raw APIs, each with conditions: Claude Managed Agents is in beta and not eligible for zero data retention or HIPAA [67, comparison table, "Data handling"]; OpenAI compares its runtime options by integration effort [68, "Compare agent runtime options"]; Google recommends a managed runtime for Python agents that need minimal operations [69, "Choose an agent runtime"]. These conditions change, so record the date checked and the provider's retirement schedule for the chosen models [70, intro].

Cost and capacity at expected volume

- Estimate cost from measured tokens per run, prices and expected traffic [71, "Managing costs"], including the variation between runs of the same task: on agentic coding tasks the dearest run cost about twice the cheapest, on average per model [72, Fig. 2]. Compare scenarios against the value of the work, and set notification thresholds and spend limits [71, "Managing billing limits"]. In Anthropic's data, agents used about 4 times the tokens of chat and multi-agent systems about 15 times [54, "Benefits of a multi-agent system"].
- Check peak requests and input and output tokens against the provider's rate limits, counting cached input as that provider does [73, "Cache-aware ITPM"]. Treat a Retry-After value as a minimum, and do not retry quota or billing errors [74, "Error mitigation", "Retrying with exponential backoff"]. On shared capacity, a 429 can mean temporary contention [75, "Resource Exhausted (429) errors"]; if the requirement needs assured capacity, assess the provider's reserved options against it [inferred].

Select models

- OpenAI: prototype with the most capable model to set a quality baseline, then try smaller models and keep them where they still meet the target [5, p. 8].
- Mannheim recommends free-model prototyping, small-model main experiments, and selective use of stronger models after estimating cost [28, slide 28]. CS329A recommends smaller, cost-effective models during development before scaling to larger models [32, "API Configuration and Setup"].
- JHU A2 compares models empirically across languages and prompting strategies [31].
- Resolution [inferred]: these optimise different budgets. If quality is uncertain, start strong to learn what is achievable; if spend is the binding constraint, start cheap and escalate. In both cases compare on the Phase 4 development set, and record quality, cost and latency per model.

Done when

One traced path runs end to end, a model table (quality, cost, latency on the development set) supports the chosen model or models, and cost and rate limits have been checked at expected volume.

Phase 7: Evaluate, diagnose and iterate

Find out why the system fails, fix the causes, and confirm that the added complexity earns its cost.

The loop (CS329T HW1, the course's "standard iterative process" [33, §§1–2]; Figure 4):

1. Run the system on the development set and read the traces.
2. Label failures by hand, grouped into error classes.
3. If you use LLM judges, check them against your hand labels before trusting them.
4. Change prompts, tools or architecture and rerun.

Supporting practice

- Mannheim: define error classes by hand on a sample, count them, then automate the analysis [28, slide 26].
- CS329T's Agent GPA checks goal fulfilment, plan quality, plan adherence, tool selection, tool calling, execution efficiency and logical consistency [34, slide 21].
- Google: evaluate end-to-end outcomes first, then inspect trajectories to diagnose [7, pp. 17–19]. Chip Huyen catalogues planning and tool failure modes and recommends testing tools independently [10].
- Separate capability evals (can it do this?) from regression evals (did a change break it?) [3].

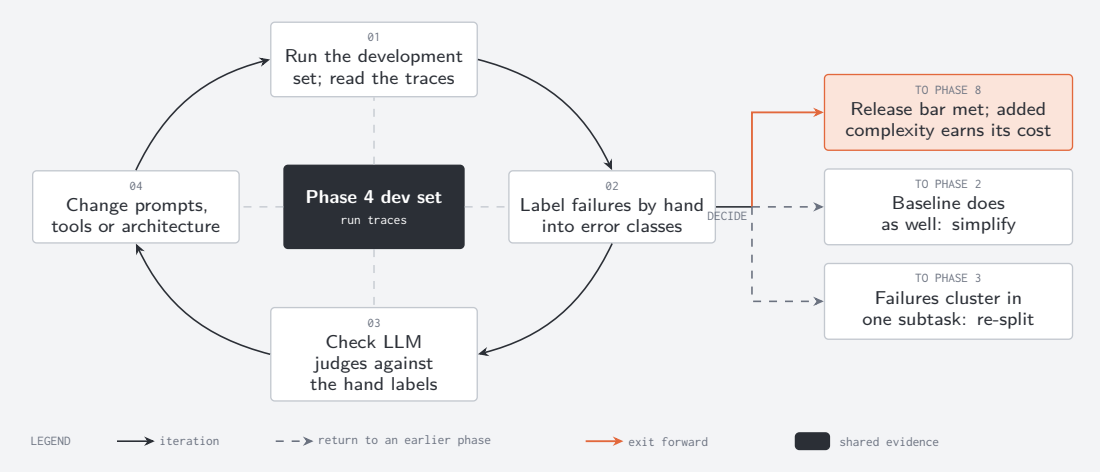


Figure 4: The Phase 7 loop. Steps 01–04 follow CS329T HW1 [33, §§1–2] and Mannheim [28, slide 26]; the exits and their conditions are my synthesis [inferred].

- Treat an LLM judge as a versioned component: record its model and version, rubric and prompt, and re-check it against held-out human labels after any change [inferred]. Control its known biases: position and verbosity [40, "LLM-as-a-judge"], and labels saying who wrote an answer, which shifted scores in both directions in one study, so hide authorship [46, §4.2]. A separate judge per dimension can help [3, step 5]. A panel of judges from different model families agreed with humans better than one larger judge in one study [47, §4.2, Table 2]; test a panel against your single judge before adopting it. Calderon et al. give a statistical test for when judge labels can replace human ones [48, §3].
- Break results down by segments chosen from the application's workflows, user groups and observed failures, and report each with its uncertainty [39, Recommendations 1–2]. An average can hide a weak segment: simulated-user success differed by dialect in one study [44, §4.3].
- Choose the fix from the diagnosis. OpenAI separates missing context (the model lacks the knowledge: improve retrieval or the context) from inconsistent behaviour (format, tone or reasoning: improve the prompt and examples, or fine-tune), notes that a problem can be both, and starts from prompting with an evaluation set [49, "LLM optimization context", "Understanding the tools"]. Try the simplest adequate intervention first and re-evaluate [inferred].
- A run without errors does not establish that its answer is correct. Notion found that a query could run cleanly and still answer the wrong question, with the meaning, routing, measurement or interpretation wrong, and that more retrieval or retries did not automatically help [12, "Why working SQL isn't enough", "Seeing what the agent actually did"].

When Phase 2 chose a bounded agent, the baseline is the strongest fixed workflow, and the comparison is the matched one planned on the screening worksheet: the same model, starting information, tools, check and limits in both arms, on cases used to design neither, so that the only difference is who selects the next step [27, Section 4].

Return paths. If the system does not beat the Phase 2 baseline by enough to justify its cost and latency, go back to Phase 2 and simplify. If failures cluster in one subtask, revisit its ownership in Phase 3 [inferred].

Done when

Error classes and counts are recorded, graders are validated, and the selected design meets the agreed success and safety thresholds. Any added complexity has a measured benefit that justifies its cost and latency; otherwise simplify and re-evaluate [inferred].

Phase 8: Harden safety and approve release

Show that the boundaries from Phases 3 and 5 hold under attack and misuse, and that the release bar is met.

- Run adversarial and misuse cases: prompt injection through tool outputs and retrieved content, attempts to exceed permissions, and inputs that should be refused. Buffalo's Lab 5 asks where autonomy helps and where it should be restricted [23]; CMU covers sandboxing and approval points [16].
- Layer guardrails: LLM-based checks, rules and moderation, added as edge cases appear; route high-risk actions and repeated failures to a human [5, pp. 24–31]. Guardrails can run as separate parallel calls [1].
- Test for leaks of sensitive data with controlled fixtures: planted records with stated authorised recipients and flows. Inspect internal channels as well as outputs. In AgentLeak's two-agent systems, messages between agents leaked more often than final outputs (68.8% against 27.2%, adversarial scenarios included), and 41.7% of all traces leaked internally while the final output stayed clean [76, Abstract; Table 11]. Include memory and cross-user cases where they apply [63, access section]. Restrictions in the system prompt can be bypassed by injection [77, intro], and an output filter for personal data is one layer, not the test [5, p. 26].
- Guardrails can miss attacks; Mannheim's safety deck covers their limits [25, pp. 17–23]. Measure their effectiveness against the system's threat model, rely on enforced permission boundaries, and explicitly assess residual risk [inferred].
- Production concerns listed by Sierra: guardrails and enforced determinism, regression testing, user simulation, staging and release, model migration and failover [36, p. 10].
- Treat safety results as hard constraints: report them separately instead of averaging them into an overall score [13, §3.5]. Where the agent's output is the final action, with no human review, set a higher reliability bar than for an assistant [13, §5, Recommendation 4].
- For agents that run unattended, use read-only credentials wherever the agent only reads, and cap what each run can spend. Anthropic suggests starting the cap at three to five times the cost of a normal run and tightening it from real numbers [11, Guardrails].
- Confirm the Phase 4 release bar, including the held-out generalisation check and regression and safety evaluations, and record who signed off. Google describes evaluation gates before release, including a manual gate for less mature teams [8, pp. 12, 17–18].

Done when

Adversarial results are recorded, every unacceptable failure from Phase 1 has a tested control, and the release bar is met and signed off.

Phase 9: Deploy gradually and maintain

Release in stages with a way back, watch the system in use, and turn what you learn into tests.

- Version prompts, tools, models and the test set together. Roll out in stages with a rollback path [8, pp. 15–16, 24]. Decide beforehand, from the Phase 1 map, what a failed canary triggers: pause, roll back, or refer to a person. Google's SRE guidance compares the canary with a control on a small set of metrics, and for non-interactive systems runs it for at least one complete unit of work [51, ch. 16]; for an agent, that unit could be a session or a job [inferred].
- Monitor quality, cost, latency and safety events in production. Monitoring complements evals; it does not replace them [3].
- For agents that run unattended, watch runtime events and also list sessions on a schedule, because an event delivery can be missed; an idle session is not necessarily finished, so read its stop reason before routing it [52, "Monitor sessions and review retention"]. Decide beforehand what happens at the spend cap [53,

"session.budget_reached"], how a failed run resumes [54, "Production reliability"], and which model and agent version each run uses [inferred].

- Triage production failures and add representative regression cases to the Phase 4 set [8, p. 24]. When choosing traces to review, mix random traces into every batch with traces targeted at the question at hand [78, "How can I efficiently sample production traces"]; user-reported failures belong in the set too [3, step 1].
- Report on a schedule: what changed, likely causes and next actions, with comparable eval results and their uncertainty, operating metrics and links to example transcripts [inferred]. If the test set changed, say so before comparing scores [inferred]. Report standard errors, and paired differences when two versions ran on the same cases [39, Recommendations 1, 4].
- Keep a register of what will need maintenance: prompt versions, pinned model identifiers with their retirement dates, deprecated parameters, and the owner of the evaluation suite; re-run the suite on any migration [inferred]. Anthropic gives at least 60 days' notice before retiring a model and advises testing replacements well before the date [70, intro]. An eval suite is a living artifact that needs clear ownership [3, step 8]. In one study of open-source projects, prompt debt was the most common LLM-specific kind of self-admitted technical debt [79, RQ2, Table 3].
- Turn recurring corrections into rules deliberately. A repeated mistake or a review finding makes a candidate [80, "When to add to CLAUDE.md"]. Decide its scope, and whether it must be enforced: "If something has to happen every time, make it a hook, a deny rule, or a managed setting" [81, "Enforce the rules that can't have exceptions"]. Test it against a baseline without the rule [82, "Build evaluations first"], and review the rule set for stale or conflicting entries [80, "Audit your instruction files"]. Rules have a cost: in one study, repository context files generally did not raise coding-agent success and raised inference cost by over 20% [83, Abstract; §4.2].
- If rules and skills must work in more than one coding harness, keep skill frontmatter to the shared standard [84, "Frontmatter"], since extra fields can cause hard errors in some tools [85, "Using skill frontmatter outside Claude Code"], and record how each harness finds its instructions. Codex concatenates AGENTS.md files from the repository root down, up to a configurable size limit [86, "How Codex discovers guidance"]; Claude Code reads AGENTS.md only in some configurations [80, "When Claude Code reads AGENTS.md"].
- Re-run the full evaluation on any model change [8, p. 18]. Sierra's development cycle is build, test, optimise, repeated [36, p. 17].
- Re-evaluate at regular intervals, not only after changes. Rabanser et al. call for periodic evaluation on evolving benchmarks, so that reliability that degrades silently is caught [13, §5 and App. A, Recommendation 1]. In deployment, apply this by regularly re-evaluating against cases and environments updated to reflect current conditions [inferred].
- Start small and validate with real users [5, conclusion]. For a scheduled agent, trigger a run by hand before relying on the schedule [11, Schedule].
- Review production traces on a schedule. At Notion, a nightly review agent records missing, stale or conflicting context, and a separate triage agent applies authorised fixes and sends ambiguous or higher-risk changes to a person; it notes that query success and rising usage are useful signals, but "neither proves accuracy" [12, "Keeping data and business context current", "Seeing what the agent actually did"].

Done when

The system is versioned, the rollout and rollback plan is tested, monitoring is live, and there is a routine for moving production failures into the test set.

Different emphases and my design choices

Question	Positions	How this process treats it
Model order	Strongest first, then downsize [5]; cheap first, then escalate [28, 32]	Choose by budget; always compare on the development set (Phase 6)
Evals or prototype first	Evals early [3]; evaluator-first project structure [38, Grading, Project Timeline]; prototype then evaluate [2]	Small set before building, grown from failures (Phase 4)
One agent or several	Single agent first [5, p. 16]; three agents for a Lean task [37]	Start simple; add agents when measured benefits justify the coordination cost [inferred] (Phase 3)
Frameworks and platforms	Direct API calls first [1]; framework essential [6]; the option with the least operational responsibility that meets the requirements [66, §3]; a managed agent runtime [69, "Choose an agent runtime"]	Specify capabilities, not a library; compare the managed options against them (Phase 6)
LLM judges	Common in course rubrics [33]; objective checks for verifiable tasks [35]	Code checks first; validate judges against humans (Phases 4, 7)
Rules: wording or enforcement	Write recurring corrections into instruction files [80, "When to add to CLAUDE.md"]; enforce rules that can't have exceptions with a hook, deny rule or managed setting [81, "Enforce the rules that can't have exceptions"]; context files often don't help [83, Abstract]	Write the rule, test it against no rule, and enforce what must always happen (Phase 9)
Failure response	A failed canary can be paused, rolled back or referred to a person [51, ch. 16]; escalate per action on retry limits and high-risk actions [5, p. 31]	The Phase 1 map decides the response for each class of failure (Phases 1, 9)
Research framing	Mechanism → manipulation → measured behaviour [22]	Research novelty is not a product requirement; mechanism-based experiments can still test design choices [inferred]

Limits of this document

- The evidence is the public material checked on 9 October 2026. Unpublished rubrics (CMU's project, Buffalo, Stanford CS146S's final project), JHU lecture PDFs over 10 MB, Google Slides and PPTX decks, and lecture videos were not read. "Not found" means not found in that material.
- The Anthropic, OpenAI and Google guides were read through extracted text and summaries; page and section numbers come from those reads. The reliability paper [13] was read in its arXiv HTML version (v3). The two practitioner posts [11, 12] each describe one system; Notion's outcome figures are self-reported.
- The version 1.10 additions come from official provider documentation and papers read in full on 2 November 2026. Each claim was checked against the full text by a reader and then by a second reviewer. Study figures are cited with their settings and do not generalise beyond them. Provider pages change; the bibliography records when each was checked.
- The phase order, the Phase 2 ladder, the gates and the resolutions of disagreements are my synthesis.

Authorship and use of AI tools

I am the sole author. I used AI tools in preparing this document and disclose their role here. *Claude* (Anthropic, through Claude Code) read the course materials and practitioner guides, and drafted and revised the text and diagrams under my direction. *Codex* (OpenAI) made an independent pass over the same sources and reviewed the citations; its corrections were applied. Claude also drafted the appendix examples and their diagrams under my direction, and Codex reviewed them against the phases and the screening worksheet. For version 1.10, Claude read the provider documentation and papers and drafted the additions, and Codex independently re-checked the evidence for each claim; claims the evidence did not support as written were narrowed or dropped. Words taken from a source are in quotation marks with the source. Practices taken from a source are cited to it, with a page

or section reference where one is available. Statements marked [inferred] are my own synthesis. I set the questions, decided what to include and am responsible for the content.

Appendix: Worked examples

Three examples follow. Each takes one task through the process:

- the Phase 1 brief;
- the screening worksheet that feeds Phase 2 [27, Section 6];
- the decisions recorded in the later phases.

A diagram in each example shows where the model chooses the next step, and where code and people keep control. The diagrams use a small subset of BPMN: a circle is an event (start, or a heavy ring for an end), a box is a task, and a diamond is a decision taken by whoever owns its lane. Every loop passes a decision in the code lane that enforces the cap. A bracketed note on a decision marks a branch the example does not describe; the drawing stops there because the example does. In all three, the strongest fixed workflow is built first. A bounded agent is kept only for the decision that the fixed workflow leaves open, and only if it wins the Phase 7 comparison. Two rules hold in every example and are not repeated in the tables. The acceptance check sits with code or a person, outside the model's control (row A3). Phase 8 ends, as in the main text, with the release bar confirmed and the release signed off. The examples are illustrations, not reports of built systems [inferred].

Example 1: a phone agent that acts during a call

A small business wants its phone answered. The agent answers questions, books, moves or cancels appointments, writes a note to the CRM, and transfers the caller when it should.

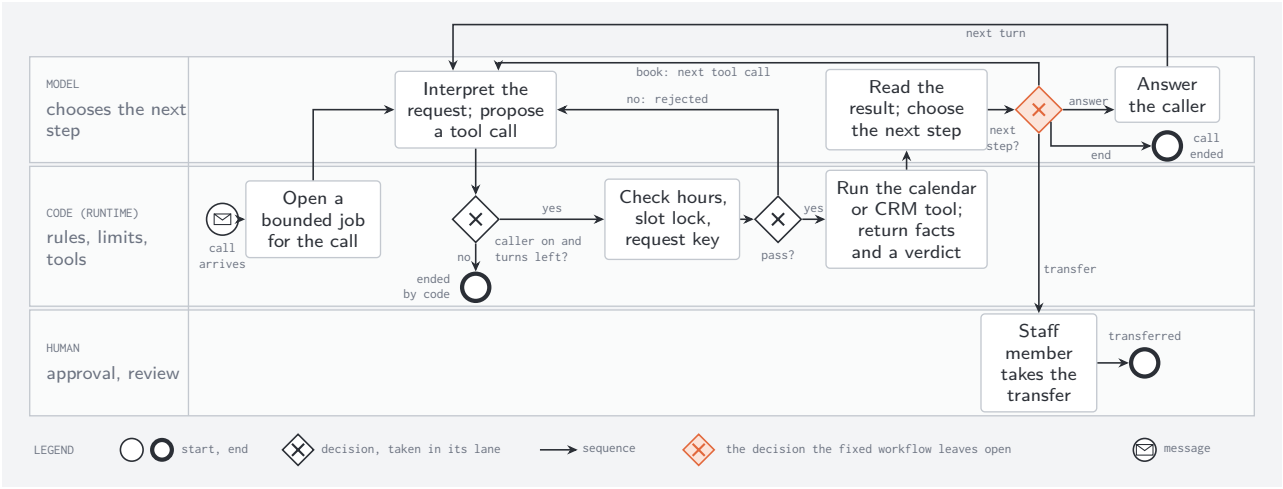


Figure 5: Example 1. Code notices the call and enforces every booking rule; the model chooses the next tool call within the call; a person takes transfers. The arrangement is an illustration [inferred].

Step	Decision
Phase 1 brief	Success is the right outcome for each call, within a response-time limit. The unacceptable failures are a double booking and an action the caller did not ask for.
Row S	Always on. The telephony service notices the call, and each call is a bounded job that ends when the caller hangs up.
Rows 0a, 0b	A keypad menu with a booking link serves callers who know what they want. One call with calendar context handles simple requests. Test both first.
Rows A1–A3	**Connecting step:** understanding what the caller wants. **Tools:** the calendar, the CRM, a knowledge base and call transfer. **Check:** each booking is compared with the caller's scripted intent, with zero duplicates, and the check is tried on known-good and known-bad calls.
Rows B1–B3	**Fixed policy:** a dialogue state machine with multiple intents, corrections, retries and transfer. **Open decision:** a request that changes mid-call and conflicts with what the calendar returns. **Worth testing** only if such calls cost more than the agent adds in running and oversight.
Phase 2 choice	The state machine first. Add a bounded agent for the open decision only if rows B1–B3 hold.
Phase 3	Code owns business hours, booking rules and the turn cap. The model proposes the next tool call. A person takes transfers (Figure 5).
Phase 4	Simulated callers for each type of request. Measure the outcome per call and the latency per turn. Report duplicate bookings separately, as a safety result.
Phase 5	Repeating a request cannot create a second booking; it returns the recorded outcome. The calendar locks each slot. Tools return facts and a verdict.
Phase 6	A traced prototype on scripted calls. Choose the model by latency per turn as well as by outcome.
Phase 7 (row B4)	The state machine against the agent, with the same model, information, tools, check and limits, on callers used to design neither.
Phase 8	Callers who try to talk the agent out of a rule.
Phase 9	One business first. Review calls on a schedule, and turn failed calls into test cases.

Example 2: an investigation agent that resolves conflicting evidence

Given a list of companies, the system finds each one's owner and cites its sources. Most entities resolve from a registry. The hard cases are renamed companies, subsidiaries and records that disagree.

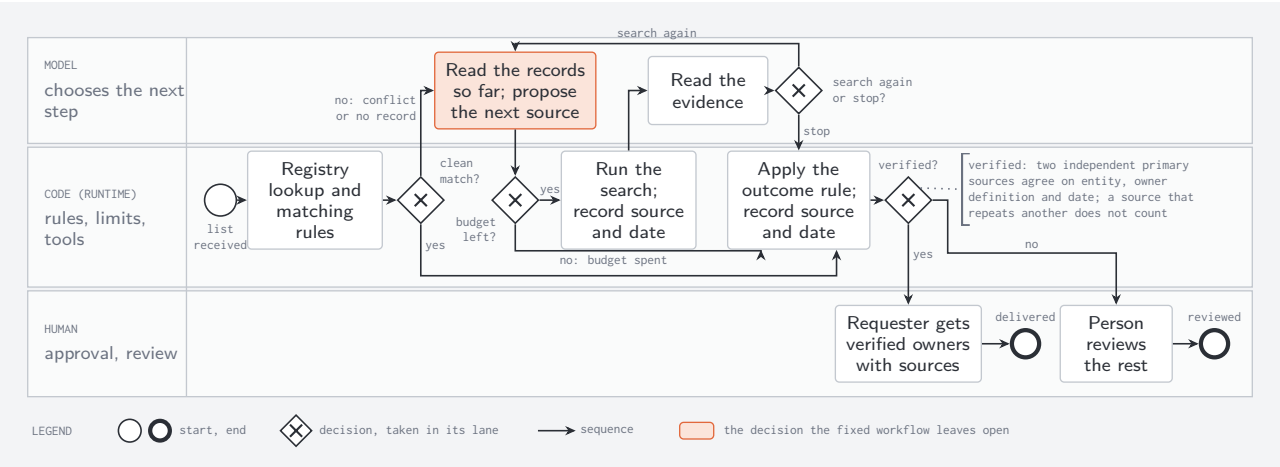


Figure 6: Example 2. Code resolves clean matches, stops the search when the budget is spent and applies the outcome rule. The model chooses where to look next only when the registry gives no clean match. A person reviews every outcome that is not verified. The arrangement is an illustration [inferred].

Step	Decision
Phase 1 brief	"Owner" is defined (legal, beneficial or ultimate parent) and dated. Success is a correct, sourced owner, or an honest "unresolved". The unacceptable failure is a confident wrong owner.
Row S	On demand. A list is handed over, and the job ends when every entity has an outcome.
Rows 0a, 0b	Registry lookups with matching rules resolve the clean cases. One call with search is the second baseline.
Rows A1–A3	**Connecting step:** deciding which conflicting record to trust and where to look next. **Tools:** registries, web search and a browser, all read-only. **Check:** a reference set of entities with verified owners, plus cases that should end "unresolved". The check is tried on known-good and known-bad outputs. Agreement between sources guides a run but does not prove the answer.
Rows B1–B3	**Fixed policy:** the registry, then a second source, then templated searches, then review. **Open decision:** which source to try when the registry gives no clean match. **Worth testing** only if the value of the cases it resolves, rare costly ones included, exceeds the agent's added model, search, oversight and failure costs.
Phase 2 choice	The fixed workflow first. Add a bounded agent for the hard cases only if rows B1–B3 hold.
Phase 3	The model proposes the next source and how to read it. Code applies the outcome rule. *Verified* needs two independent primary sources, such as a registry filing and a document not copied from it, that agree on the entity, the definition of owner and the date; a page that repeats another source does not count. The other outcomes are *insufficient evidence*, *conflict* and *budget spent*, and a person reviews all three (Figure 6).
Phase 4	Accuracy against known owners, the unresolved rate and the cost per entity.
Phase 5	Every result carries its source and retrieval date, and each search counts against the budget.
Phase 6	A traced prototype of the fixed workflow on a few known entities. Choose the smallest model that reads the records correctly.
Phase 7 (row B4)	The fixed workflow, with its branches and retries, against the agent. Both get the same model, information, tools and check, and the same limits on searches, model calls, time and total cost, on reference cases used to design neither. A run without errors can still name the wrong owner, so read the traces.
Phase 8	Pages that carry injected instructions.
Phase 9	Monitor the unresolved rate and the cost per entity, and add reviewed cases to the reference set.

Example 3: a planning run that adapts from performance data

Each week the system proposes what content to make next, from the business's own material and last period's results. A fixed workflow then produces each approved item. The candidate agent is the planning run: within it, the model chooses which analysis to run next before it proposes the slate.

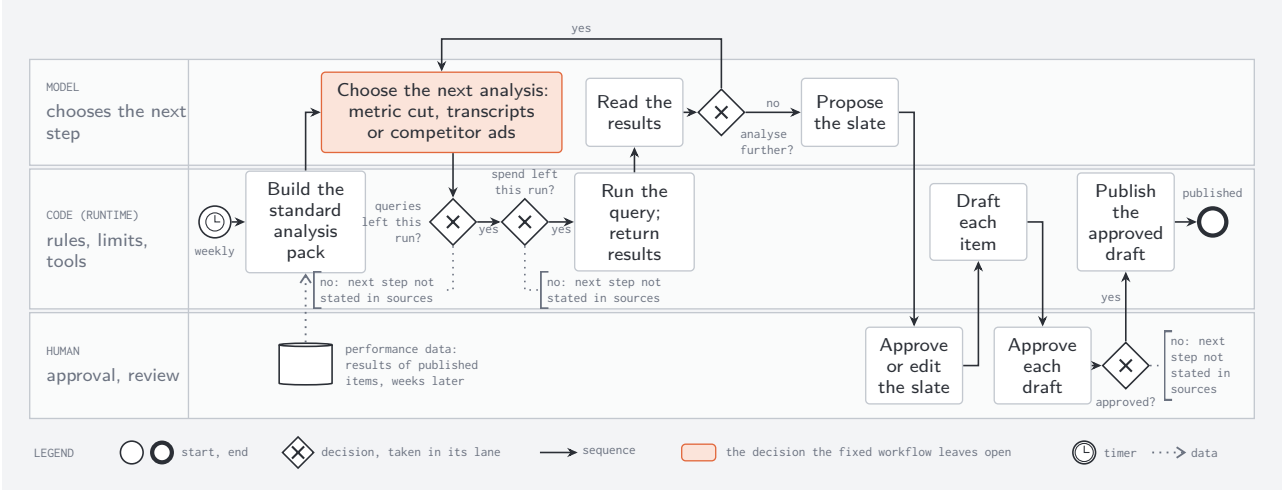


Figure 7: Example 3. The agent sits inside one step of a fixed weekly workflow. Within the planning run it chooses the next analysis. A person approves the slate and then each draft; production stays fixed. Results mature weeks later and inform later runs, not the current one. The arrangement is an illustration [inferred].

Step	Decision
Phase 1 brief	Success is a slate the owner approves, and leads that grow over the quarter. The unacceptable failures are publishing without approval and false claims.
Row S	Always on, on a schedule. Code starts a weekly planning job that ends with a proposed slate.
Rows 0a, 0b	A rule ("more of what performed best") and a multi-armed bandit over topics. One call given the standard analysis pack.
Rows A1–A3	**Connecting step:** working out why something performed and choosing the next angle. **Tools:** analytics queries, transcript search and competitor-ad search. **Check:** within the run, query results; after it, approval of the slate and leads weeks later. The approval is tried on slates already known to be good and bad.
Rows B1–B3	**Fixed policy:** the analysis pack plus drill-downs triggered by set anomalies. **Open decision:** which further analysis would explain a result the drill-downs do not. **Worth testing** only if the expected gain in leads exceeds the agent's added running, oversight and failure costs, and there is enough history to estimate that gain.
Phase 2 choice	A fixed workflow for production, and the bandit or one call for planning. Add a bounded agent for the planning run only if rows B1–B3 hold. Phase 2 warns that weak feedback can misdirect adaptation.
Phase 3	A person approves the slate, and each draft before it is published. Code caps queries and spend per run (Figure 7).
Phase 4	A factual check on drafts and the approval rate of slates before launch, then leads, reported with their delay.
Phase 5	Read-only analytics access for planning; publishing tools only in the production workflow.
Phase 6	A traced prototype of one planning run on past weeks. Choose the smallest model whose slates pass approval.
Phase 7 (row B4)	The fixed policy, with its drill-downs, against the agent. Both get the same model, information, tools, check and limits, on weeks used to design neither. Exposure is comparable, leads are measured over the same window, differences between weeks and channels are controlled, and the precision is estimated before the lead comparison is read.
Phase 8	Injected text in transcripts and competitor pages.
Phase 9	Watch for the loop chasing views at the expense of leads, and re-evaluate on a schedule.

References

[1] Anthropic, "Building Effective Agents". https://www.anthropic.com/engineering/building-effective-agents

[2] Anthropic, "Writing effective tools for agents". https://www.anthropic.com/engineering/writing-tools-for-agents

[3] Anthropic, "Demystifying evals for AI agents" (Jan 2026). https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

[4] Anthropic, "Effective context engineering for AI agents". https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[5] OpenAI, "A practical guide to building agents". https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf

[6] Google/Kaggle, "Introduction to Agents" (May 2026). https://www.kaggle.com/whitepaper-introduction-to-agents

[7] Google/Kaggle, "Agent Quality" (May 2026). https://www.kaggle.com/whitepaper-agent-quality

[8] Google/Kaggle, "Prototype to Production" (May 2026). https://www.kaggle.com/whitepaper-prototype-to-production

[9] LangChain, "Thinking in LangGraph". https://docs.langchain.com/oss/python/langgraph/thinking-in-langgraph

[10] Chip Huyen, "Agents" (Jan 2025), section "Agent Failure Modes and Evaluation". https://huyenchip.com/2025/01/07/agents.html

[11] L. Martin and C. Avilla (Anthropic), "Building effective agent automations", claude.dev blog (8 Oct 2026). https://claude.dev/blog/building-effective-agent-automations/

[12] D. Thomas (Notion), "How we built a personal data scientist for every Notion employee", Notion blog (8 Oct 2026). https://www.notion.com/blog/how-we-built-a-personal-data-scientist-for-every-notion-employee

[13] S. Rabanser, S. Kapoor, P. Kirgis, K. Liu, S. Utpala, A. Narayanan, "Towards a Science of AI Agent Reliability", ICML 2026; arXiv:2602.16666v3 (2 Jun 2026). https://arxiv.org/abs/2602.16666v3

[14] CMU 11-768, lecture 2 slides (tool use). https://www.cmu-agents.com/slides/lecture-02-tool-use.pdf

[15] CMU 11-768, lecture 5 slides (planning). https://www.cmu-agents.com/slides/lecture-05-planning.pdf

[16] CMU 11-768, lecture 13 slides (agent safety). https://www.cmu-agents.com/slides/lecture-13-agent-safety.pdf

[17] Stanford CS146S (Fall 2026), week 2 assignment. https://github.com/mihail911/modern-software-dev-assignments/blob/master/week2/assignment.md

[18] Stanford CS146S (Fall 2026), week 3 assignment. https://github.com/mihail911/modern-software-dev-assignments/blob/master/week3/assignment.md

[19] Northeastern, "Tools decide, or the model decides?" (week 3b). https://agenticllms.github.io/week3b-slides.html

[20] Northeastern, week 4a slides (workflows, routers, agents). https://agenticllms.github.io/week4a-slides.html

[21] Northeastern, DADA1 project specification, §9 "Getting started". https://github.com/agenticllms/lecture-files/blob/main/DADA1/DADA1_SPEC.md#9-getting-started

[22] Northeastern, research-project resources, "The rule". https://agenticllms.github.io/research.html

[23] University at Buffalo CSE 410/510 (Fall 2026), Labs 3–5 and Final Team Project. https://cse.buffalo.edu/courses/cse410/2026_fall/cse410ae/

[24] Universität Mannheim IE685, deck 4 (LLM agents and tool use). https://www.uni-mannheim.de/media/Einrichtungen/dws/Files_Teaching/Large_Language_Models_and_Agents/FSS2026/IE685_LA_04_LLMAgentsAndToolUse.pdf

[25] Universität Mannheim IE685, deck 8 (security and safety). https://www.uni-mannheim.de/media/Einrichtungen/dws/Files_Teaching/Large_Language_Models_and_Agents/FSS2026/IE685_LA_08_SecurityAndSafety.pdf

[26] J. A. Arroyo, *Learning to Build Agentic Systems: An instructional compilation guide built from public university courses* (October 2026). Document 1 of this set; indexes the courses cited here. File `guide/agentic_guide.pdf`.

[27] J. A. Arroyo, *Screening Tasks for Agentic Automation* (v0.9.3, November 2026). Document 3 of this set. File `task_screening/task_screening.pdf`.

[28] Universität Mannheim IE686, "Introduction to Student Projects" (IE686_LA_07). https://www.uni-mannheim.de/media/Einrichtungen/dws/Files_Teaching/Large_Language_Models_and_Agents/FSS2026/IE686_LA_07_IntroStudentProjects.pdf

[29] CU Boulder CSCI 7000-005, project proposal. https://danny.cs.colorado.edu/courses/csci7000-005_Sp26/projectProposal.html

[30] JHU EN.601.727, Assignment 3 README (§§1.2–3.3). https://github.com/machine-programming/assignment-3

[31] JHU EN.601.727, Assignment 2. https://github.com/machine-programming/assignment-2

[32] Stanford CS329A (Autumn 2025), Homework 3 notebook. https://github.com/stanford-cs329a/cs329a-homework3-fall2025-public/blob/main/homework3.ipynb

[33] Stanford CS329T (Fall 2025), HW1 handout. https://web.stanford.edu/class/cs329t/homeworks/CS329T_HW1_Fall2025.pdf

[34] Stanford CS329T, Lecture 3 slides (Agent GPA). https://web.stanford.edu/class/cs329t/slides/Lecture%203%20--%20CS%20329T%20Fall%202025.pdf

[35] UC Berkeley Agentic AI (Fall 2025), "LLM Agent Evaluations & Project Overview". https://rdi.berkeley.edu/agentic-ai/slides/LLM%20Agent%20Evaluations%20%26%20Project%20Overview.pdf

[36] UC Berkeley Agentic AI (Fall 2025), "Practical Lessons from Deploying Real-World AI Agents" (Sierra). https://rdi.berkeley.edu/agentic-ai/slides/Practical_Lessons_AI_Agents.pdf

[37] UC Berkeley Advanced LLM Agents (Spring 2025), Lab 2 instructions, "Recommended Implementation". https://drive.google.com/file/d/1_8ASqnVR99BTiKd-I7g8Ov7-UiXsJeHW/view

[38] UC Berkeley Agentic AI (Fall 2025), course page, "Grading" and "Project Timeline". https://rdi.berkeley.edu/agentic-ai/f25

[39] E. Miller (Anthropic), "A statistical approach to model evaluations" (Nov 2024). https://www.anthropic.com/research/statistical-approach-to-model-evals

[40] OpenAI, "Evaluation best practices", API docs (checked 2 Nov 2026). https://developers.openai.com/api/docs/guides/evaluation-best-practices

[41] Anthropic, "Define success criteria and build evaluations", Claude Platform docs (checked 2 Nov 2026). https://platform.claude.com/docs/en/test-and-evaluate/develop-tests

[42] C. Li et al., "A Scoping Review of LLM-as-a-Judge in Healthcare and the MedJUDGE Framework", arXiv:2604.25933. https://arxiv.org/abs/2604.25933

[43] Google, Agent Development Kit, "User Simulation" (checked 2 Nov 2026). https://adk.dev/evaluate/user-sim/

[44] P. Seshadri et al., "Lost in Simulation: LLM-Simulated Users are Unreliable Proxies for Human Users in Agentic Evaluations", arXiv:2601.17087. https://arxiv.org/abs/2601.17087

[45] X. Zhou et al., "Mind the Sim2Real Gap in User Simulation for Agentic Tasks", COLM 2026; arXiv:2603.11245. https://arxiv.org/abs/2603.11245

[46] S. Chae et al., "Self- and Other-Labels Induce Bidirectional Bias in LLM Judges", arXiv:2608.18091. https://arxiv.org/abs/2608.18091

[47] P. Verga et al., "Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models", arXiv:2404.18796. https://arxiv.org/abs/2404.18796

[48] N. Calderon, R. Reichart, R. Dror, "The Alternative Annotator Test for LLM-as-a-Judge", arXiv:2501.10970. https://arxiv.org/abs/2501.10970

[49] OpenAI, "Optimizing LLM Accuracy", API docs (checked 2 Nov 2026). https://developers.openai.com/api/docs/guides/optimizing-llm-accuracy

[50] NIST, AI RMF Playbook, "Manage" (checked 2 Nov 2026). https://airc.nist.gov/airmf-resources/playbook/manage/

[51] Google, *The Site Reliability Workbook*, ch. 16 "Canarying Releases" (2018). https://sre.google/workbook/canarying-releases/

[52] Anthropic, "Review and monitor agent sessions", Claude Managed Agents docs (beta; checked 2 Nov 2026). https://platform.claude.com/docs/en/managed-agents/secure-deployment-monitoring

[53] Anthropic, "Managed Agents tutorial: production setup", Claude Cookbook (8 Apr 2026). https://platform.claude.com/cookbook/managed-agents-cma-operate-in-production

[54] Anthropic, "How we built our multi-agent research system" (13 Jun 2025). https://www.anthropic.com/research/multi-agent-research-system

[55] Anthropic, "When to use multi-agent systems (and when not to)" (23 Jan 2026). https://claude.com/resources/articles/building-multi-agent-systems-when-and-how-to-use-them

[56] N. Tomašev, M. Franklin, S. Osindero (Google DeepMind), "Intelligent AI Delegation", arXiv:2602.11865. https://arxiv.org/abs/2602.11865

[57] M. Cemri et al., "Why Do Multi-Agent LLM Systems Fail?", NeurIPS 2025 Datasets and Benchmarks; arXiv:2503.13657v3. https://arxiv.org/abs/2503.13657v3

[58] Anthropic, "Create custom subagents", Claude Code docs (checked 2 Nov 2026). https://code.claude.com/docs/en/sub-agents

[59] Google PAIR, *People + AI Guidebook*, "User Needs + Defining Success". https://pair.withgoogle.com/chapter/user-needs/

[60] Google PAIR, *People + AI Guidebook*, "Explainability + Trust". https://pair.withgoogle.com/chapter/explainability-trust/

[61] Google PAIR, *People + AI Guidebook*, "Feedback + Control". https://pair.withgoogle.com/chapter/feedback-controls/

[62] T. Trinh et al. (Scale AI), "HiL-Bench: Do Agents Know When to Ask for Help?", arXiv:2604.09408v4. https://arxiv.org/abs/2604.09408v4

[63] Anthropic, "Using agent memory", Claude Managed Agents docs (beta; checked 2 Nov 2026). https://platform.claude.com/docs/en/managed-agents/memory

[64] Google, Agent Development Kit, "State" (checked 2 Nov 2026). https://adk.dev/sessions/state/

[65] Y. Margalit et al., "Governed Shared Memory for Multi-Agent LLM Systems", arXiv:2606.24535. https://arxiv.org/abs/2606.24535

[66] Microsoft Learn, Cloud Adoption Framework, "AI adoption" strategy (updated 15 Oct 2026). https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ai/strategy

[67] Anthropic, "Claude Managed Agents overview" (beta; checked 2 Nov 2026). https://platform.claude.com/docs/en/managed-agents/overview

[68] OpenAI, "Agents", API docs (checked 2 Nov 2026). https://developers.openai.com/api/docs/guides/agents

[69] Google Cloud Architecture Center, "Choose your agentic AI architecture components" (reviewed 21 Apr 2026). https://docs.cloud.google.com/architecture/choose-agentic-ai-architecture-components

[70] Anthropic, "Model deprecations", Claude Platform docs (checked 2 Nov 2026). https://platform.claude.com/docs/en/about-claude/model-deprecations

[71] OpenAI, "Production best practices", API docs (checked 2 Nov 2026). https://developers.openai.com/api/docs/guides/production-best-practices

[72] L. Bai et al., "How Do AI Agents Spend Your Money? Analyzing and Predicting Token Consumption in Agentic Coding Tasks", arXiv:2604.22750v3. https://arxiv.org/abs/2604.22750v3

[73] Anthropic, "Rate limits", Claude API docs (checked 2 Nov 2026). https://platform.claude.com/docs/en/api/rate-limits

[74] OpenAI, "Rate limits", API docs (checked 2 Nov 2026). https://developers.openai.com/api/docs/guides/rate-limits

[75] Google Cloud, Vertex AI, "Standard PayGo" (dynamic shared quota; checked 2 Nov 2026). https://docs.cloud.google.com/vertex-ai/generative-ai/docs/dynamic-shared-quota

[76] F. El Yagoubi, G. Badu-Marfo, R. Al Mallah, "AgentLeak: A Benchmark for Internal-Channel Privacy Leakage in Multi-Agent LLM Systems", IEEE Access (2026); arXiv:2602.11510. https://arxiv.org/abs/2602.11510

[77] OWASP GenAI Security Project, "LLM02:2025 Sensitive Information Disclosure". https://genai.owasp.org/llmrisk/llm022025-sensitive-information-disclosure/

[78] H. Husain and S. Shankar, "AI Evals: Everything You Need to Know" (FAQ). https://hamel.dev/blog/posts/evals-faq/

[79] A. Aljohani et al., "PromptDebt: A Comprehensive Study of Technical Debt Across LLM Projects", arXiv:2509.20497. https://arxiv.org/abs/2509.20497

[80] Anthropic, "How Claude remembers your project", Claude Code docs (checked 2 Nov 2026). https://code.claude.com/docs/en/memory

[81] Anthropic, "Best practices for Claude Code", Claude Code docs (checked 2 Nov 2026). https://code.claude.com/docs/en/best-practices

[82] Anthropic, "Skill authoring best practices", Claude Platform docs (checked 2 Nov 2026). https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices

[83] T. Gloaguen et al. (ETH Zurich), "Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?", arXiv:2602.11988v3. https://arxiv.org/abs/2602.11988v3

[84] Agent Skills, "Specification" (checked 2 Nov 2026). https://agentskills.io/specification

[85] Anthropic, "Extend Claude with skills", Claude Code docs (checked 2 Nov 2026). https://code.claude.com/docs/en/skills

[86] OpenAI, "Custom instructions with AGENTS.md", Codex docs (checked 2 Nov 2026). https://learn.chatgpt.com/docs/agent-configuration/agents-md